



量子コンピューティング I

#4 Grover アルゴリズム

情報工学科 准教授 佐藤貴彦

第 4 回の目標

- 今日の講義を通じてできるようになること:
 1. Grover アルゴリズムを 4 ステップに分解できる
 - ✓ オラクル / Diffusion / 反復 / 測定
 2. 問題から Grover 用 オラクルを設計できる
 - ✓ ライツアウトを「Yes/No 判定」に落とし込む
 3. 反復回数を計算・最適化できる
 - ✓ なぜ k_{opt} で止めるか、どう計算するか

中間レポートの実装を 自分の手で組む 準備が整う

復習: オラクル型アルゴリズムの 4 ステップ

■ #3で学んだ、量子アルゴリズムのテンプレート:

Step 1 重ね合わせ H で全状態を等しく準備



Step 2 位相をいじる オラクルが条件成立時に位相 -1



Step 3 **干渉** **振幅が解に集まる仕組み (Sec.I-IIで詳しく)**



Step 4 測定 高確率で解が観測される

Groverでは
ここを $O(\sqrt{N})$ 回 反復する

■ #3では、2つの量子アルゴリズムを学んだ

Deutsch: 古典 2 → 量子 1 (定数倍の高速化)

Grover: 古典 $O(N)$ → $O(\sqrt{N})$ (本物のクエリ複雑度の優位性)

#4ではGroverをの挙動を理解し、ユースケースに落とし込んで実装する

復習：#3 で取り扱ったこと

■ #4ではこれらを前提知識としてGroverに応用します

1. 位相キックバック

CX の 2 つの機能、標的からのZ 情報の逆流 → 「位相を制御する」ための実装手段

2. クエリ複雑度

関数呼び出し回数で量子優位性を定量化 → 「 $O(\sqrt{N})$ は何回反復するか」の基礎

3. 量子算術

Toffoli + CNOT で任意の論理関数を組める → オラクル設計の部品

全体構成 (6 部 + まとめ)

Sec.I Grover の構成要素

→ #3のデモをベースに

Sec.II 反復回数の本質

→ $O(\sqrt{N})$ の繰り返しとは

Sec.III ライツアウト

→ YES/NO問題に落とし込む

Sec.IV ナイーブ版な設計

→ qubit 数・シミュレーションの壁

Sec.V アルゴリズムの改良提案

→ ゲームの性質で探索空間削減

Sec.VI オラクル設計の一般化

→ 中間レポートの準備

まとめ 次回からはNISQ 編

Sec.I

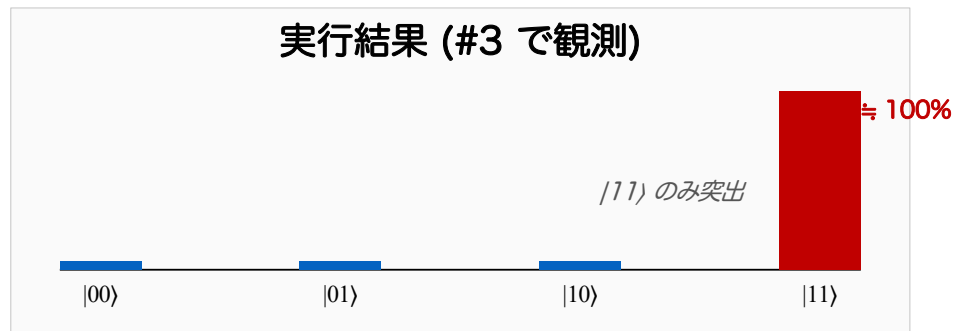
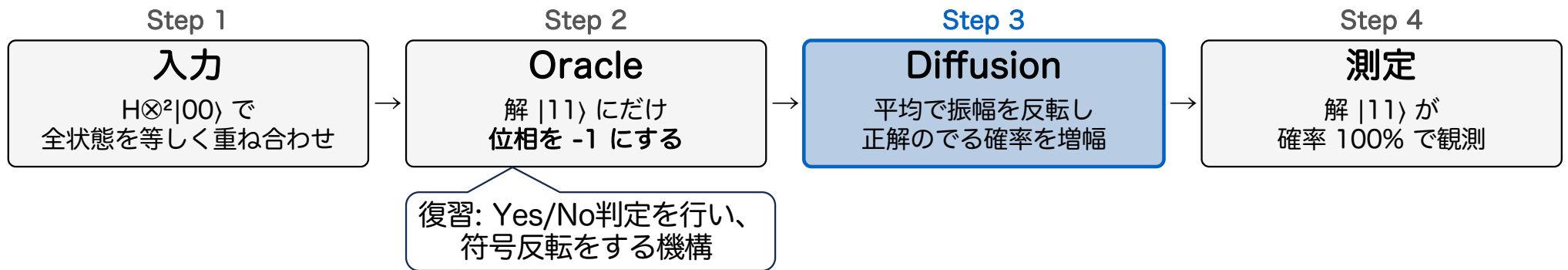
Grover の構成要素

Step 1: 重ね合わせ | Step 2: 位相を操作 | Step 3: 干渉 | Step 4: 測定

まずは、Step 2と3 の機能を見ていこう

前週 Grover デモの復習 (2qubit, N=4)

このNだとStep 2, 3は
反復なしで良い (後述)



いま知りたいこと:

Diffusion で
振幅がどう増幅されているか

「位相を使った干渉」とは？ 平均で振幅を反転するとは？

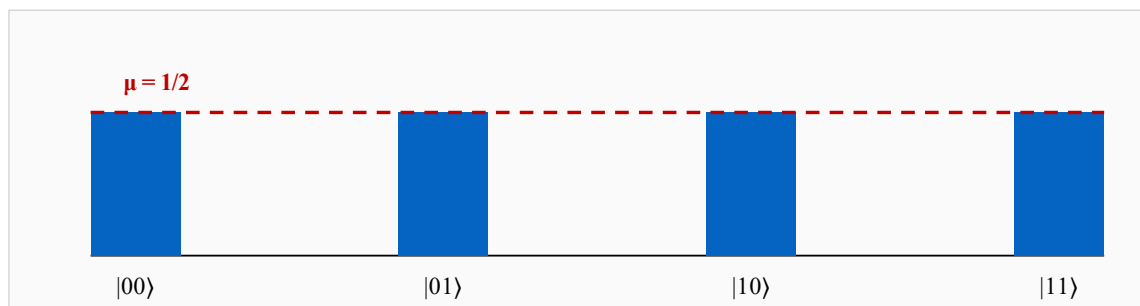
そもそも「振幅の平均」とは？

- 各状態 $|x\rangle$ には確率振幅 (複素数) がある:

$$|s\rangle = \sum a_x |x\rangle \quad a_x \text{ が状態 } |x\rangle \text{ の振幅}$$

- ✓ 「振幅の平均」 = 全状態の振幅の算術平均値:

$$\mu = (1/N) \sum a_x$$



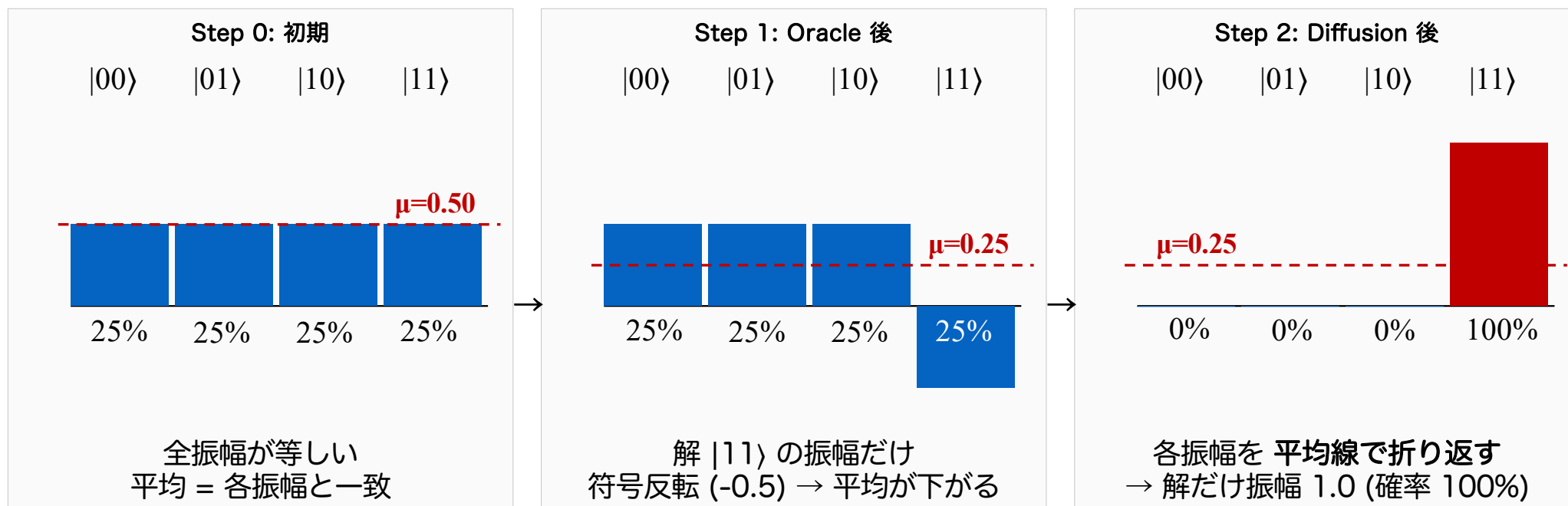
ボルンの規則
振幅の絶対値の2乗が
測定確率であった

- 初期状態: 全振幅が等しい
✓ それぞれ 25% ▶ 振幅は $1/2$
→ 平均は、各振幅と一致している

何かが起きて一部の振幅が変わると、平均線が動く

(注: Grover の枠組みでは振幅は実数で扱える、複素振幅の話は副読本第 8 章)

振幅は各ステップでどう動く？ (N=4 の場合)



Diffusion の機能 = 平均についての反転
各振幅を平均線を軸に折り返す操作

Step 2 で $|11\rangle$ が振幅 1.0 に: 平均から最も遠かったため (式での確認は次スライド + Notebook の Slide 10 連動セル)

Step 2: 「平均に対する反転」を式で確認 (N=4 の手計算)

- Diffusion 後の “ 状態_x ” の振幅:

$$a'_x = 2\mu - a_x \quad (\mu = \text{振幅の平均})$$

この操作に対応する
量子回路は後に解説する

- N=4 の手計算 (Step 1 → Step 2):

Step 1 後: 振幅 = (1/2, 1/2, 1/2, -1/2) → $\mu = 1/4$

Step 2 ($a' = 2\mu - a$ を各振幅に適用):

$$|00\rangle: 2(1/4) - 1/2 = 0$$

◀0 に潰れる

$$|01\rangle: 2(1/4) - 1/2 = 0$$

$$|10\rangle: 2(1/4) - 1/2 = 0$$

$$|11\rangle: 2(1/4) - (-1/2) = 1$$

◀振幅 1 → 確率 $|1|^2 = 100\%$

「平均から最も遠かった振幅」が一気に伸びる

Notebook の Slide 10 連動セル で、式とstate vector実装が一致することを確認しよう

中間整理： Diffusion とは何だったか

■ Slide 9 / 10 で $N=4$ の具体例を体感した。ここで一旦立ち止まって整理する：

ここまでで分かったこと

- Diffusion = 平均についての反転
(Slide 9 の図で体感)
- 公式 $a' = 2\mu - a$
(Slide 10 の手計算で確認)
- $N=4$ では 1 反復で振幅 1.0 に到達

次の疑問点

- 量子回路でどう書く？
 - ✓ 「平均」は古典的な計算操作
 - ✓ 量子では H, X, CZ などの限られたゲートしか使えない
 - ✓ これらの組合せで「平均に対する反転」を実現する必要がある

次のスライドで、Diffusion を 3 段階に分解する

(線形代数の言葉では「 $2|s\rangle\langle s| - I$ 」、これも「平均に対する反転」の別表現)

Step 2: 「平均反転」をどう実装するか? (Diffusion 3 段階)

- 動機: 「平均についての反転」をしたい / でも、量子回路でどう書く?

$$H^{\otimes n} \rightarrow X^{\otimes n} \rightarrow CC..CZ \rightarrow X^{\otimes n} \rightarrow H^{\otimes n}$$

“CC..CZゲート” =
“制御 制御 .. Zゲート”
✓ 標的にZをかけるToffoliに相当

段階 1 $H^{\otimes n}$

基底を変える操作

- ✓ 重ね合わせ状態 $|s\rangle$ を、 $|0\dots 0\rangle$ が「平均」の役割を担う基底へ変換

制御制御..NOTゲート
(3~量子ビットゲート)

段階 2 $X^{\otimes n} \rightarrow CC..CZ \rightarrow X^{\otimes n}$

新しい基底で「 $|0\dots 0\rangle$ への反転」を実行

- ✓ X で 0 と 1 を入れ替え / CC..CZ で $|1\dots 1\rangle$ に位相 -1 / X で戻す $\rightarrow |0\dots 0\rangle$ 以外の振幅が反転

段階 3 $H^{\otimes n}$

元の基底に戻す

- ✓ 結果: 元の世界での「平均についての反転」が実装される

数学的根拠 (線形代数として):

Diffusion = $2|s\rangle\langle s| - I$ (元の基底) $\rightarrow H^{\otimes n}$ で挟むと: $H^{\otimes n} (2|s\rangle\langle s| - I) H^{\otimes n} = 2|0\dots 0\rangle\langle 0\dots 0| - I$
 $\rightarrow |s\rangle$ が $|0\dots 0\rangle$ に写るので、段階 2 が実行するのは「 $|0\dots 0\rangle$ への反転」、段階 3 で元の基底での平均反転になる

原点への射影操作、などとも言われる

Notebook の Slide 12 連動セル で H 基底変換 ($|s\rangle \rightarrow |0\dots 0\rangle$) を実装 / 詳細は副読本第 8 章

Step 1: Oracle の実装 (#3 復習)

- OracleはYes/Noを判定する機能 (#3). どうやって実装する？

✓ 復習: CX には 2 つの機能が合った

機能 ① 制御NOT

制御が $|1\rangle$ なら標的を反転

機能 ② 位相キックバック

標的が $|-\rangle$ なら制御に位相 -1

- “Oracleにおける符号反転”は 以下の 2 パターンの実装が有力:

(A) アンシラ $|-\rangle$ + CC..CX (X の固有値 -1 を利用) ← 本日採用 (機能②の応用)

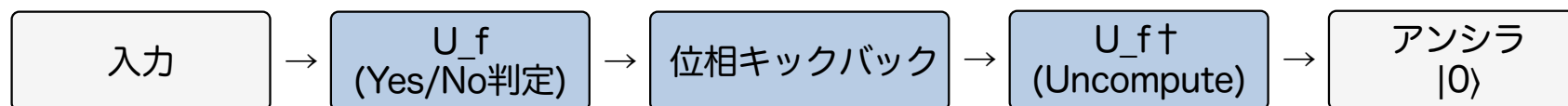
(B) アンシラ $|1\rangle$ + CC..CZ (Z の固有値 -1 を利用)

(復習) アンシラは
補助量子ビット

Oracleでは、Toffoliゲートを利用して、「条件を満たす入力にだけ位相 -1 を乗せる」

Step 1: Uncomputation の必然性 (アンシラを管理する)

- アンシラはOracleの終了時に初期状態に戻っている必要がある
 - ✓ Uncomputation (逆計算) と呼ばれる操作で戻ることができる
- Oracleの流れ
 - ✓ U_f で判定を行うが、アンシラを $|0\rangle$ に戻すため、意外と複雑な構成になる



✗ もつれたまま

アンシラと入力がつれた状態
→ Diffusion の対称性が壊れる
→ 振幅増幅が機能しない

○ $|0\rangle$ に戻す

入力レジスタだけが
「位相が反転した状態」
→ Diffusion が正しく機能

[Notebook の Slide 14 連動セル](#) で「Uncompute なし vs あり」の対照実験 (なしだと振幅増幅が壊れる)

詳細は副読本第 7 章末尾

Sec.I まとめ | Groverアルゴリズムの中身を覗いた

- Sec.I で取り扱ったこと
 - Grover の 4 フェーズ (前週デモの復習)
 - Diffusionについて
 - ✓ 平均に対する反転 (Slide 9, 10)
 - ✓ 実装 = $H^{\otimes n} \rightarrow X^{\otimes n} \rightarrow CC..CZ \rightarrow X^{\otimes n} \rightarrow H^{\otimes n}$ の 3 段階 (Slide 12)
 - Oracleについて
 - ✓ Yes/No判定を位相に書き込む機構
 - ✓ 実装 = 位相キックバック + アンシラ + Uncomputation (Slide 13-14)

Next Section

最適な反復回数

OracleとDiffusionは
 $O(\sqrt{N})$ 回繰り返す必要がある

[N=4 では 1 回の実行で機能したが、 N が大きくなると何が変わるのだろうか？](#)

Sec.II

反復回数の本質

N=4 は 1 回の実行で正解に辿り着いたが、大きい N ではどうなるだろうか？

Step 1: 重ね合わせ | **Step 2: 位相を操作** | **Step 3: 干渉** | Step 4: 測定

Step 2 (符号反転) ► Step 3 (増幅) ► Step 2 ► Step 3.. と 最適な回数だけ 反復させる必要がある

N が大きくなると、何が変わる？

- N=4 では OracleとDiffusion を 1回 実行すると 100% 正解が得られた (Sec. I)
 - ✓ どんなNでも1回で良いなら超絶加速だが・・
 - ✓ 実際は 符号反転▶増幅▶符号反転▶増幅.. と繰り返していくことになる
- Nが大きくなると、OracleとDiffusionを $O(\sqrt{N})$ 回 実行すると 高確率で正解が得られる
 - ✓ クエリ複雑度で証明された量子加速
 - ✓ 実は、ナイーブに $\sqrt{N}$ 回 繰り返す とうまくいかない・・

N = 4	→ 2 回? (1回で成功したはずだが?)
N = 16	→ 4 回?
N = 64	→ 8 回?
N = 512	→ 23 回?

意外とピーキーな
アルゴリズム

実は、 $\sqrt{N}$ より少し小さい最適反復回数 k_{opt} があり、それを超えると成功確率が下がる

(次のスライドでは、発展的な内容だが 2次元平面の話 と 幾何学的根拠を見る)

発展 | 最適繰り返し回数と 2 次元平面・・・？

難しいので吹き出しを確認すればOK

- 入力空間は 2^n 次元 ($N = 2^n$ 個の基底状態)だが、それらを以下の 2 集合に分ける:

解の集合 (M 個)

$|w\rangle = (1/\sqrt{M}) \sum |x\rangle$
(x が解)
(今回の例: $M=1$, 解は $|11\rangle$)

非解の集合 (N - M 個)

$|s'\rangle = (1/\sqrt{N-M}) \sum |x\rangle$
(x が非解)
(今回の例: $|00\rangle, |01\rangle, |10\rangle$ をまとめたもの)

Groverアルゴリズムは
この 2 集合の確率を操作する
(解の確率を増幅し、非解の確率を下げる)

- Oracle と Diffusion は「解どうしの比」「非解どうしの比」を変えない:

具体例で確認 (N=4、Slide 10 の手計算と連動):

Step 0 (初期)

$|w\rangle$: 全部 1/2 (1 個)
 $|s'\rangle$: 全部 1/2 (3 個)
→ 等しい振幅

Step 1 (Oracle 後)

$|w\rangle$: -1/2 (1 個)
 $|s'\rangle$: 全部 1/2 (3 個)
→ $|s'\rangle$ の比は保たれた

Step 2 (Diffusion 後)

$|w\rangle$: 1.0 (1 個)
 $|s'\rangle$: 全部 0 (3 個)
→ $|s'\rangle$ の比は保たれた

- $|w\rangle$ と $|s'\rangle$ の 2 つの軸だけで 解と非解の状態 $|s\rangle$ が完全に決まる (N 次元 → 2 次元)

$$|s\rangle = \alpha |w\rangle + \beta |s'\rangle \quad (\alpha^2 + \beta^2 = 1)$$

Groverアルゴリズムは
 α を上げ、 β を下げることを目指している

Notebook の Slide 18 連動セル (NEW) で $N=8$, $M=2$ のケースを検証 ($N=4$, $M=1$ と同じ挙動を示す)

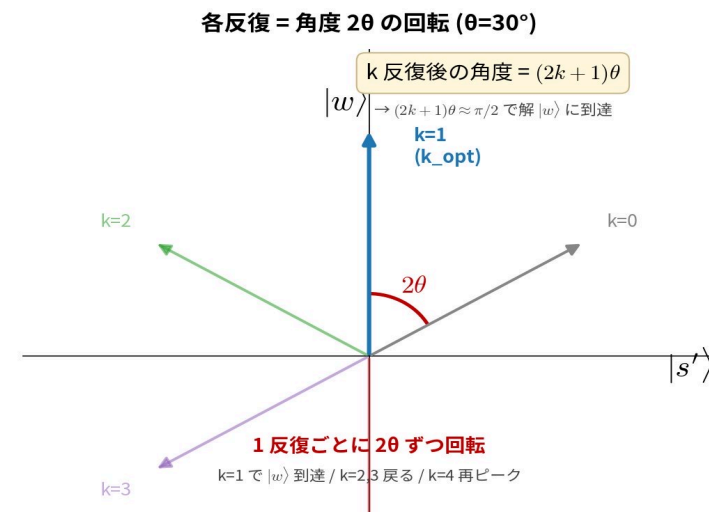
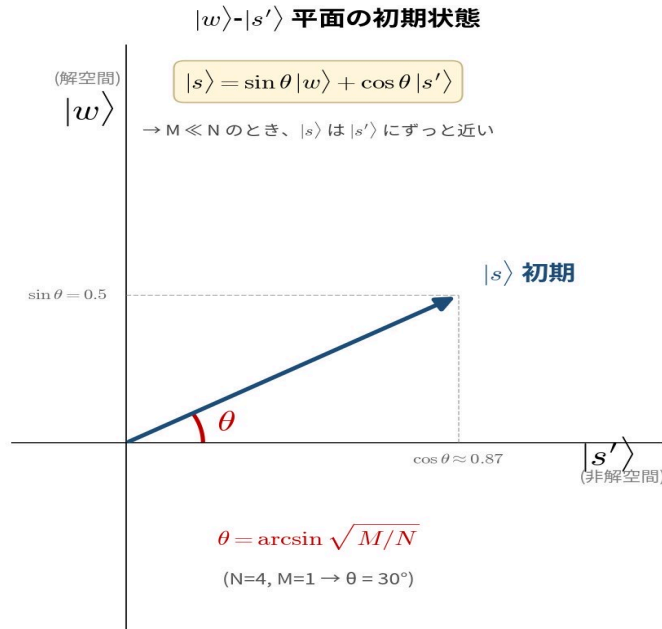
発展 | 最適反復回数 k_{opt} と 2D 平面回転

背景: 2 つの反射の合成は回転になる (高校幾何?)

Oracle = $|s'\rangle$ 軸に対する反射

Diffusion = 元の $|s\rangle$ に対する反射 $\rightarrow$ 1 反復 = $|s\rangle$ は角度 2θ 回転する

$|s\rangle$ と $|w\rangle$ が最接近するところで止める = 最適反復回数



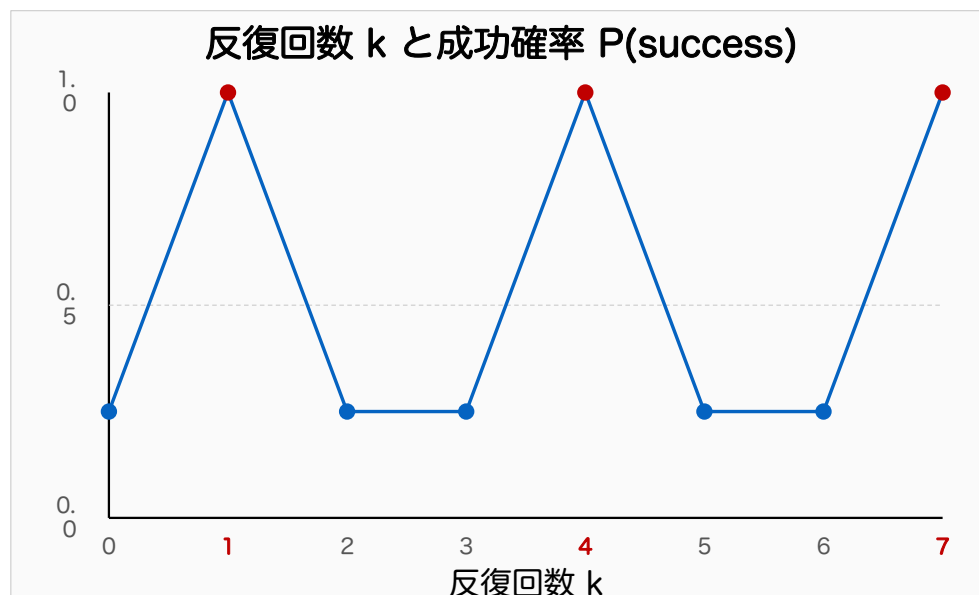
最適な反復回数は M と N で決まる: $k_{\text{opt}} = \lfloor \pi / (4 \cdot \arcsin \sqrt{M/N}) \rfloor$

$N=4, M=1$ で $k_{\text{opt}}=1$ になる

k_opt を超えて反復してしまうとどうなる？

- 成功確率は減っていったしまう・・・(2D 平面で角度が $\pi/2$ を超えるため)

✓ N=4 の場合 ($k_{\text{opt}} = 1$) で確認:



観察:

k = 0 → 0.25 (初期)

k = 1 → $\hat{=}$ 1.0 (k_{opt} = 最初のピーク)

k = 2 → $\hat{=}$ 0.25 (初期に戻る)

k = 3 → $\hat{=}$ 0.25 (まだ低い)

k = 4 → $\hat{=}$ 1.0 (再びピーク)

...

→ ピークは $k = 1, 4, 7, \dots$ (周期 3)

→ 実用上は $k_{\text{opt}} = 1$ で止めるのが安全

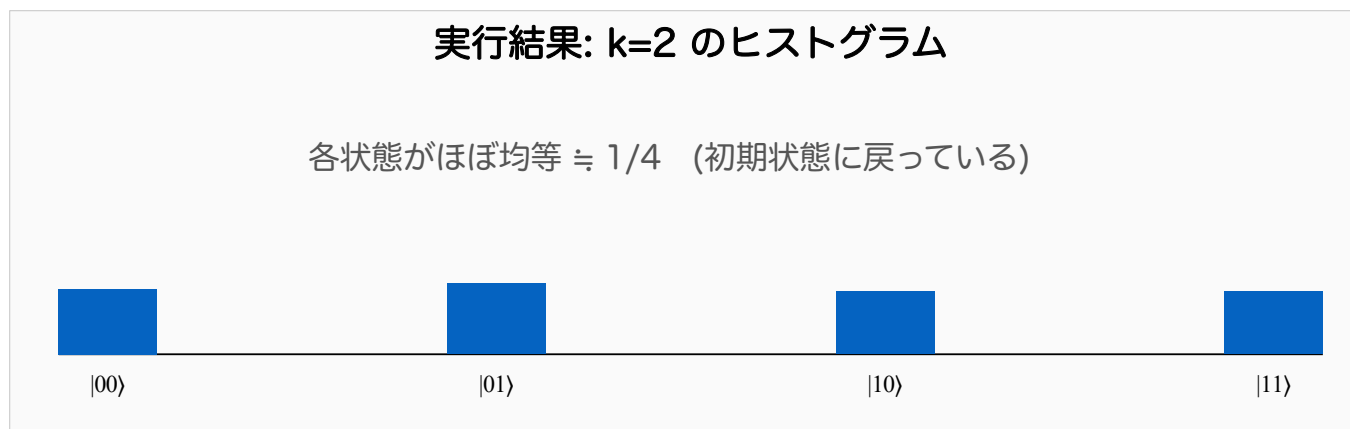
Notebook の Slide 20 連動セル で確認しよう

2D 平面で角度が $\pi/2$ を行ったり来たり: 回転が続いているだけ (Slide 19 参照)

演習 1 | N=4 で 2 反復してみよう (体感)

1 min

- Notebook の Slide 21 連動セル を実行してみよう
 - ✓ 前ページのグラフから $k=2$ では確率 ≈ 0 になるはず



k_{opt} を 1 つ超えただけで、増幅前と同じ状態に戻ってしまう!

重要なこと:

- Grover は反復回数を増やせばいいわけではない
- k_{opt} で止めることが重要 (「正しい止め時」を知る必要がある)

Sec. II まとめ | Grover が動く条件

■ ここまでで分かったこと

条件 1: Oracle が「Yes/No 判定」を位相 -1 でマーキングできること

条件 2: 反復回数 k が $k_{\text{opt}} (\doteq \pi/4 \cdot \sqrt{N/M})$ に近いこと

→ この 2 条件を満たせば、解の振幅が増幅される

■ ライツアウトでこの 2 条件を満たせるか？

条件 1 のチェック

ライツアウトの「全消灯」を判定する量子回路を組めるか？

→ Sec.III-V で Oracle 設計を体験

条件 2 のチェック

3×3 ライツアウトの k_{opt} は？ $N=2^9=512$, $M=1$

→ 17 反復

→ Sec.IV-V で実装、動作確認

Next Section

ライツアウトを古典で解いてから、量子化する方法を考えよう

Sec.III

ライツアウト

まずは古典的に問いてみよう

Step 1: 重ね合わせ | Step 2: 位相をいじる | Step 3: 干渉 | Step 4: 測定

(本節はテンプレートの「適用先」を考える)

ルール: ライツアウト

- ・ ボタンを押すと、そのボタンと上下左右の 4 つが反転
- ・ 全消灯すればクリア

0	1	2
3	4	5
6	7	8

初期状態

ボタン 4 を押す
(1, 3, 5, 7も反転)



0	1	2
3	4	5
6	7	8

クリア！



日本ではタカラ社から
1995年に発売された

オリジナルは 5×5、本講義では探索空間の都合で 3×3 で扱う (シミュレータで動かしやすい範囲)

演習 2 | 3 分例題チャレンジ

0	1	2
3	4	5
6	7	8

盤面: lights = [0,1,0,1,1,0,1,1,0] → 1, 3, 4, 6, 7 が点灯

どのボタンを押せば全消灯できる?

(自分の頭で考えてみよう)

全てのボタンは高々 1 回押せば良い
総当たりでも 512 通りで解ける

解答: ボタン 1, 3, 5, 8 を押す

0	1	2
3	4	5
6	7	8

初期

→

0	1	2
3	4	5
6	7	8

push 1

→

0	1	2
3	4	5
6	7	8

push 3

→

0	1	2
3	4	5
6	7	8

push 5, 8
→ 全消灯

各 push が反転するボタン:

push 1 → ボタン 0, 1, 2, 4 が反転
 push 3 → ボタン 0, 3, 4, 6 が反転
 push 5 → ボタン 2, 4, 5, 8 が反転
 push 8 → ボタン 5, 7, 8 が反転

rule

じつは線形代数の問題に落とし込んで
 ガウスの消去法で解くこともできる
 (総当たりで強いGroverを使う必然性はない)

3×3 ライツアウトの特性 + 量子化の動機

■ 3×3 ライツアウトの特性:

- 解が一意に定まる (5×5 だと複数解 or 解なし)
- 初期配置に関わらず必ず解が存在
- 各ボタンは高々 1 回押せば足りる (冪等性)
- 押す順番は結果に影響しない (可換)

■ 探索空間: $2^9 = 512$ 通り、解は 1 つ ► **M=1, N=512**

- 決定問題なのでオラクル U_f が組める
- Grover で解けるはず!

■ 量子化の動機:

- 教育的: 具体例で Grover を組める体験
- スケーラビリティ: 5×5 (2^{25}), 7×7 (2^{49}) への拡張
- 副読本第 8 章: NP 問題への応用、SAT 等

Next Section

Oracleを設計する

ナイーブな実装から考えよう

Sec.IV

ナイーブなOracle設計と動作確認

データ構造 → オラクル → 動作確認 → 改良への発想

Step 1: 重ね合わせ | **Step 2: 位相を操作** | Step 3: 干渉 | Step 4: 測定



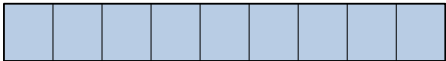
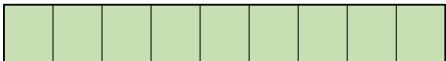
ここをどうやるのか考える

まずはデータ構造を考える

入力 (問題): lights = [0,1,0,1,1,0,1,1,0]

出力 (解): solution = [0,1,0,1,0,1,0,0,1] (押すボタン: 1, 3, 5, 8)

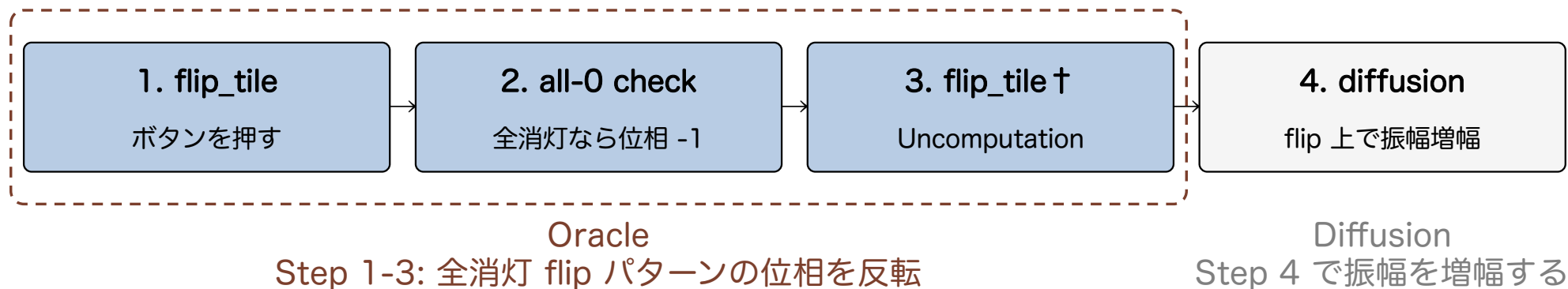
レジスタ構成:

flip	(9 qubit)		← 押すボタンの全パターンを重ね合わせ
tile	(9 qubit)		← 盤面の状態をエンコード
oracle	(1 qubit)		← 位相反転用アンシラ $ -\rangle$
auxiliary	(7 qubit)		← all-0 check 用 (Uncompute が必要)

合計: $9 + 9 + 1 + 7 = 26$ qubit あれば十分

AerSimulator では 26 qubit が現実的上限内で動作する (実行時間は数十分、本講義では事前計算結果を提示予定)

Oracleの構成と、反復操作



Step 1-4 を k opt 回繰り返した後、flip を測定すると、解が得られるはず

flip_tile の構造 (CNOT による論理回路)

python

```
def flip_tile(qc, flip, tile):  
    # ボタン i を押すと、ボタン i 自身と上下左右が反転  
  
    # i = 0: 押す → tile[0,1,3] が反転  
    qc.cx(flip[0], tile[0])  
    qc.cx(flip[0], tile[1])  
    qc.cx(flip[0], tile[3])  
  
    # i = 1: 押す → tile[0,1,2,4] が反転  
    qc.cx(flip[1], tile[0])  
    qc.cx(flip[1], tile[1])  
    qc.cx(flip[1], tile[2])  
    qc.cx(flip[1], tile[4])  
  
    # ... 9 個全て (略) → 次スライドで自分で実装!
```

ボタン 0 を押すと…

0	1	2
3	4	5
6	7	8

tile[0, 1, 3] が反転

(#3 半加算器の発想と同じ:
量子で論理演算を組む技術)

演習 3 | ナイーブ版を一緒に組み立てよう

7 min

Notebook で実装する関数:

★ 演習 ★ 1. `flip_tile(qc, flip, tile)`

ボタンを押す操作を CNOT で表現 / ヒント: ボタン i を押す $\rightarrow$ `tile[i]` と上下左右が反転

以下は提供:

- 2. `lights_out_oracle(qc, ...)` 全消灯の確認: `tile` が全 0 なら `oracle` に位相 -1
- 3. `naive_diffusion(qc, ...)` 振幅増幅: flip 9 qubit 上の標準 Diffusion
- 4. `build_naive_grover(...)` 上記を統合して Grover ソルバを組む完成形

なぜノートブックの実装でうまくいくのか考えてみよう

隣接関係 (ヒント):

0	1	2
3	4	5
6	7	8

push 0

0	1	2
3	4	5
6	7	8

push 4

0	1	2
3	4	5
6	7	8

push 8

[Notebook の Slide 32 連動セル で実装 \(ヒント L1-L3 折りたたみ式\)](#)

アルゴリズムは同じでも、実装で実行時間が変わる

- ナイーブ版には 2 つの実装があり、論理は同じだが実行時間は桁違い:

26 qubit 版 (ここまでの実装)

flip(9) + tile(9) + oracle(1)
+ auxiliary(7, Toffoli ツリー)

all-0 check が段階的に見える
→ 構造がわかりやすい (教材としてはベター)

実行時間 (k=17):
30 分超 (講義中に実行不可?)



論理は同じ/
Qubit数が違う

20 qubit 版 (シミュレータ最適化)

flip(9) + tile(9) + oracle(1)
+ auxiliary(1, mcx 直接)

AerSimulator が内部で
最適化を発動できる

実行時間 (k=17):
12 秒 (講義中に確認可能)

- 量子アルゴリズム は 古典と同様に 論理 (数学) ・ 実装 (エンジニアリング) の双方が重要
- 両者とも $k_{\text{opt}} = 17$ 、99.8% で同じ解 (押すボタン 1, 3, 5, 8、qiskit表記 100101010) が観測される

講義中の動作確認は 20 qubit 版で行おう / 中間レポート オプション 1+ では別の改良 (行依存性等) で更なる高速化を狙う

高速版を実行すると・・・ (20 qubit, k=17)

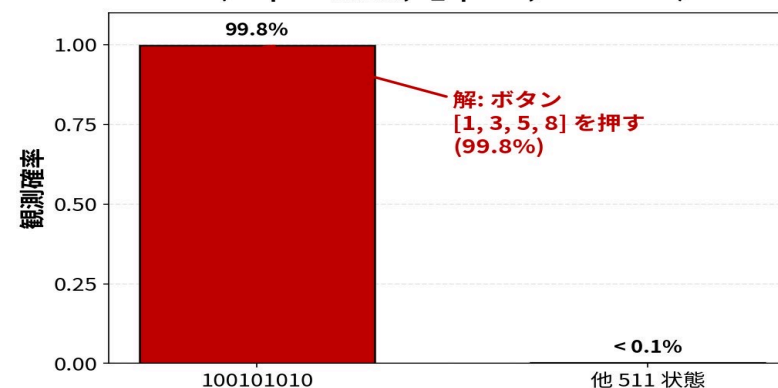
python

```
# ナイーブ版 Grover ソルバを実行
lights = [0,1,0,1,1,0,1,1,0]
qc = build_naive_grover(
    lights,
    num_iterations=17 # k_opt
)

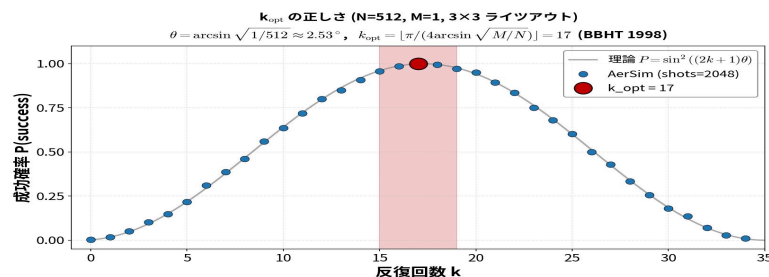
counts = run_aer(qc, shots=2048)
```

実行時間: 数十分 (26 qubit AerSim) / Notebook では mcx 直接版 (20 qubit / k=17 / 12 秒) でライブ実行可

ナイーブ Grover 3×3 ライツアウト 実行結果
(20 qubit 高速版, k_opt=17, shots=2048)



- 余力があれば、k_opt の正しさ (N=512、Slide 20 の振動を 3×3 ライツアウトで)も検証しよう



問題をYes/No問題に落とし込み、
Slide 22 の 2 条件 (Oracle が組める / 反復数 k_opt) を満たせば
Grover は実際に機能することが確認できた

Next Section: さらなる効率化を目指す

Sec.V

改良への発想: ゲームの性質を活用する

探索空間を $2^9 \rightarrow 2^3$ に削減する発想

| 実装は中間レポート オプション 1+ で |

Step 1: 重ね合わせ | **Step 2: 位相を操作** | Step 3: 干渉 | Step 4: 測定



問題固有の構造を用いたOracle効率化

3×3 ライツアウトの隠れた性質 / 行依存性

- 1 行目の押下パターンを決めると、2 行目以降は一意に決まる

0	1	2
3	4	5
6	7	8

例：1 行目を押した後
(残り点灯あり)

→

0	1	2
3	4	5
6	7	8

2 行目では点灯している
マスの真下を押す

→

0	1	2
3	4	5
6	7	8

3 行目も同様に決まる
(全消灯されなければ
1 行目のパターンを変える)

- この性質を活用すると、「1 行目をどう押すか」だけが自由度 となる

探索空間: $2^9 = 512 \rightarrow 2^3 = 8$ (64 分の 1 に削減される)

→ 次スライドで改良版の構造と結果を確認 (実装は中間レポート オプション 1+ へ)

改良版の構造 + 結果 (実装は中間レポートへ)

ナীব版 (再掲)

flip 9 qubit ($H^{\otimes 9}$)
 tile 9 qubit
 oracle 1 qubit
 auxiliary 7 qubit

合計 26 qubit, 最適反復回数は17回

改良版 (構造のみ提示)

flip 9 qubit ($H^{\otimes 3}$ 1 行目のみ重ね合わせ)
 tile 9 qubit
 oracle 1 qubit
 auxiliary 1 qubit (大幅削減)

合計 20 qubit, 反復回数は・・・?

■ 反復構造 (発想):

U: flip₁ → inv₁ → flip₂ → inv₂ → flip₃ → Oracle (lights_out) → U[†] (逆順 Uncompute) → Diffusion
 (1 行目を決める → inv₁/inv₂ が CNOT で 2/3 行目を反転 (制御として実装) → Oracle → Uncompute → 3 qubit 空間で振幅増幅)

■ 結果はこうなるはず・・・:

 他 7 個の状態 (各 < 0.1%) → ナীব版と同じ精度を、より少ない反復で実現

実装は中間レポート オプション 1+ で挑戦

ヒント:

- ・ 行依存性の活用 (本スライドで示唆)
 - ・ 列依存性 / 対称性も自分で発見してみよう
- その他、自分で発見した改良も組み込んでみよう

Sec.V まとめ | ゲーム性質 → 一般化

Sec.V で取り扱ったこと:

- ナイーブ版を動かした (Slide 34)
 - ✓ Qubit数削減が実行時間に直結
- 問題の構造に注目 (行依存性 = 1 行目で全決定)
 - ✓ 探索空間を $2^9 \rightarrow 2^3$ (64 倍削減)
- 改良版の構造を確認した
 - ✓ 実装は中間レポートで自分の手で完成

Next Section

Sec.VI Grover の汎用性と中間レポート

ライツアウトで掴んだ「設計の作法」を、他の問題に応用する:

- ・ 設計の 5 ステップを抽出
- ・ 他の Yes/No 問題への適用例
- ・ 中間レポート オプション 2 として挑戦可

Groverを他の Yes/No 問題に応用する

オラクル設計テンプレート (5 ステップ)

- 1 問題を Yes/No 判定に落とす**
「 $f(x) = 1 \Leftrightarrow x$ が解」となる関数 f を定義
ライツアウト: $f(\text{flip}) = 1 \Leftrightarrow \text{flip}$ で押すと全消灯
- 2 判定をチェックする量子回路を組む**
 $f(x)$ を計算する回路を組み、結果を auxiliary qubit に書く
ライツアウト: $\text{flip_tile} \rightarrow \text{all-0 check}$ で auxiliary が $|1\rangle$ になる
- 3 auxiliary を $|-\rangle$ で保持して位相反転**
判定結果が $|1\rangle$ なら、 $|-\rangle$ アンシラ経由で位相 -1 がキックバック
 $X \rightarrow H$ で auxiliary を $|-\rangle$ に、CCX で位相反転
- 4 判定回路を Uncompute**
Step 2 の逆操作で auxiliary を元に戻す (もつれ解消)
ライツアウト: flip_tile を再度実行 (CNOT は自身の逆)
- 5 Diffusion で振幅増幅**
重ね合わせを作った qubit 上で標準 Diffusion (H, X, CC..CZ, X, H)
ライツアウト: $\text{flip 9 qubit (ナイブ)}$ or 3 qubit (改良) 上で実施

アルゴリズムの汎用性は非常に高い

適用可能な題材例 (Yes/No に落とせる問題)

たとえば、以下のような問題はすべて、Yes/No 判定に落とせる → Grover で解ける:

数独 (9×9 や 4×4)

$f(x) = 1 \Leftrightarrow x$ がルールを全て満たす

Oracle: 9 行 + 9 列 + 9 ブロックの check を AND

SAT (CNF 充足可能性)

$f(x) = 1 \Leftrightarrow x$ が全てのクローズを満たす

Oracle: 各クローズの OR → 全クローズの AND

グラフ k-彩色

$f(x) = 1 \Leftrightarrow$ 隣接頂点が異なる色

Oracle: 全エッジで「両端の色が異なる」を AND

ナンバーリンク

$f(x) = 1 \Leftrightarrow$ 全てのペアが連結

Oracle: 経路の連結性を check (条件が複雑、副読本に詳細)

【規模感の参考: 3-SAT 5 変数の場合】

問題例:

5 変数 $x_1..x_5$, 6 節
(各節は 3 リテラルの OR)

$(x_1 \vee \neg x_2 \vee x_3) \wedge \dots$

レジスタ構成:

- ・ 入力 (変数): 5 qubit
 - ・ 節 check: 6 qubit
 - ・ Oracle aux: 1 qubit
 - ・ auxiliary: 数 qubit
- 合計 約 15-20 qubit

探索空間と反復回数:

- ・ $N = 2^5 = 32$
- ・ $M =$ 充足解の数
(問題により 1~数個)
- ・ k_{opt} :
 $M=2$ なら ≈ 3 、 $M=1$ なら ≈ 4

→ AerSimulator で軽快に動く規模 / 他の小規模題材も同程度 (グラフ 3-彩色 5 ノード等)
(中間レポート オプション 2 で挑戦可・次スライドで進め方を提示)

中間レポート (選択制)

以下のどちらかを選択して取り組む:

オプション 1+: 改良版実装

ナイーブ版を改良し、qubit数削減 or 効率化を実現:

実装すべき改良 (いずれか or 複数):

- ・ 行依存性 (本編 Slide 36-35、 $2^9 \rightarrow 2^3$)
- ・ Diffusion の別実装で深度削減
- ・ Oracle 部分の構造最適化
- ・ その他、自分で発見した改良

進め方:

- ・ Slide 42 の 5 ステップに従う

最低条件:

- ・ ナイーブ版より少ない qubit 数で完成
- ・ 動作確認 (ヒストグラムで解の振幅が突出)

オプション 2: 自由問題設計

任意の Yes/No 判定問題を設計、Grover で解く:

題材例 (Slide 40 参照):

- ・ 数独 (4×4 など、qubit 内に収まる規模)
- ・ SAT (3変数~5変数)
- ・ グラフ k-彩色 ($n=4$)
- ・ ナンバーリンク等

進め方:

- ・ Slide 42 の 5 ステップに従う
- ・ 副読本第 8 章「Grover の応用例」も参考に

評価軸: オプション 1+ → 改良アイデア + 創意性 + 比較 / オプション 2 → 問題妥当性 + オラクル設計 + 動作確認
オプション 2の方が自由度が高いため、高得点を狙いやすいです

レポートの進め方(5 ステップ + 留意事項)

1	題材選定	PDF: オプション1は改良案、オプション2は問題 + Yes/No 条件	(なし)
2	レジスタ設計	PDF: input/state/oracle/aux の qubit 数と役割	Notebook: QuantumRegister 宣言
3	オラクル設計 (Slide 39 の 5 ステップ)	PDF: 数式 or 擬似コードで Encode → Check → Uncompute	Notebook: build_oracle 関数
4	Diffusion + 反復回数	PDF: k_{opt} の計算根拠	Notebook: 標準 Diffusion or カスタム
5	動作確認	PDF 2 頁目: 結果と考察	Notebook: AerSim 実行 + ヒストグラム

提出物: PDF (1-2 頁、設計+結果+考察) + Notebook (動作再現可能)

△ オプション 2 で気をつけてほしいこと

1. 計算量に注意

データ qubit が増えると指数的に計算時間が増える / k が大きすぎても同じ → 15-25 qubit 程度に題材を選ぶ

2. 正解の数 M を見積もる

M が事前に分からないと k_{opt} が決まらない (Slide 19 参照) → 題材選定時に M を推定 or 既知の問題を選ぶ

3. 失敗した場合の原因分析を歓迎

動かなかったレポートも評価対象 → 原因分析 (k 不適 / Oracle のバグ / Uncompute 漏れ等) ができれば深い学習になる

参考 | チェックリスト

☐ 1. 問題設定

- ・ 元の問題 (あれば最適化問題等)
- ・ Yes/No 化の操作 (例: 「最短経路」 → 「総距離 D 以下の経路は存在するか?」)
- ・ なぜこの形式で意味があるか

☐ 2. 探索空間 N と解の個数 M

- ・ $N = 2^{\text{(入力 qubit 数)}}$
- ・ M (既知の問題なら値、推定なら根拠)
- ・ 推定が困難な場合はその理由も説明

☐ 3. 最適な反復回数 k_{opt}

- ・ 公式 $k_{\text{opt}} = \lceil \pi / (4 \cdot \arcsin(\sqrt{M/N})) \rceil$ の計算
- ・ 計算結果の数値 (例: 「 $N=32, M=2 \rightarrow k_{\text{opt}} = 3$ 」)

☐ 4. オラクル設計

- ・ 数式 or 擬似コードで Encode → Check → Uncompute
- ・ アンシラを使う場合は Uncomputation の有無も明示

☐ 5. 実装と動作確認

- ・ AerSimulator 実行結果 (ヒストグラム、観測確率)
- ・ 期待値 ($P = \sin^2((2k+1)\theta)$) との比較
- ・ 失敗した場合は原因分析 (前スライド留意事項 3 参照)

注: Diffusion は標準実装でよい (本編 Slide 12 参照)、カスタム実装の場合のみ PDF に追記
(チェックリストは最低限の構成要素 / 創意工夫は別途評価)

提出形式・締め切り

提出物 (2 点セット):

1. PDF レポート

1-2 ページ / 設計の意図と工夫を記述

2. Jupyter Notebook

実行可能なコード / AerSimulator で動作確認済み

締め切り: [5/14 23:59]
提出先: [K-LMS]

評価観点: 設計の妥当性 + 動作確認の丁寧さ + 工夫の独自性
(コード量より、設計の質を重視)

ヒント・参考資料

進め方の指針:

1. 古典で解いてみる (Slide 25-26 でやったように)
2. Yes/No 判定の形に整形 (Slide 39 Step 1)
3. qubit 数の見積もり (シミュレータで走るか確認)
4. 判定回路を量子化 (CX/CCX/X で論理式を実装)

参考資料:

- ・ 副読本 第 8 章 (Grover アルゴリズムの導出と汎用性)
- ・ Qiskit Textbook "Grover's Algorithm" (Qiskit 公式)
- ・ Nielsen & Chuang Ch.6 (古典)
- ・ #3 v2.2 配信物 (位相キックバック・量子算術)

実機ならどうなる？ + 期末レポート予告

- 基礎編では量子回路を AerSimulator (ノイズ無しの理想系シミュレータ) で動かした
 - 今日のライツアウトソルバを実機 (IBM Quantum) で走らせると...

△ 現状の Grover はノイズに完全に埋もれる

成功確率がノイズと区別できないレベルまで低下してしまう

- しかし、アルゴリズム + ミドルウェア の改善で、実機でも問題が解けるようになる:

NISQアルゴリズム

- ・ ノイズ耐性のある構成
- ・ VQE, QAOA, 量子NNなど

ミドルウェア

- ・ 量子コンパイラ
- ・ 確率的エラー緩和 など

来週からの NISQ 編 では、実機を使用してこれらのノウハウを学びます